

Modified Genetic Algorithm for Solving n-Queens Problem

Jalal eddin Aghazadeh heris
Faculty of Mathematics and Computer Science
Allameh Tabataba'i University
Tehran, Iran
j.aghazadeh@st.atu.ac.ir

Mohammadreza Asgari Oskoei
Faculty of Mathematics and Computer Science
Allameh Tabataba'i University
Tehran, Iran
oskoei@atu.ac.ir

Abstract— Genetic algorithm is applicable to a wide range of constraint satisfaction problems such as n-queens problem. In the absence of specialized solution for a particular problem, genetic algorithm would be efficient. But holism and random choices cause problem for genetic algorithm in searching large state spaces. So, the efficiency of this algorithm would be demoted when the size of state space of the problem grows exponentially. In this paper, we attempt to cover this weakness by using local search algorithm like minimal conflicts algorithm. Minimal conflicts algorithm is trying to provide partial view for genetic algorithm by locally searching the state space. This may cause genetic algorithm to take longer steps toward the solution. Modified genetic algorithm, is the result of collaboration between genetic algorithm and minimal conflicts algorithm. Comparing the results of applying standard genetic algorithm and modified genetic algorithm on n-queens problem in section VI, indicates the amount of performance improvement.

Keywords—genetic algorithm; minimal conflicts algorithm; n-queens problem;

I. INTRODUCTION

Genetic algorithm is member of computational method's family which is inspired by evolution [1]. Performance of genetic algorithm is flexible enough to make it applicable to a wide range of problems, such as the problem of placing N queens on N by N chessboard in order that no two queens can attack each other which is known as 'n-Queens problem'.

Genetic algorithm like many of heuristic algorithms, does not guarantee of finding solution because choosing starting point of search and taking steps toward solution have been carried out randomly. In problems like n-Queens that its state space grows exponentially, starting point of search is directly related to the probability of finding solution. Furthermore, n-Queens problem is a member of NP problem's family [2][3][4].

One of the first attempts to use genetic algorithm for solving n-Queens problem has been made by Turner and his colleagues in [5]. In [6] Wu and Meng with the idea of hybridizing genetic algorithm and local search algorithms, try to increase the efficiency of genetic algorithm in mentioned problem. Also in [7], Boiikovic and his colleagues applied genetic algorithm to n-Queens in parallel form. In [4], Sharma and his colleague show the notable performance of Cuckoo search on n-Queens

problem. Also in [8], Amooshahi and his colleagues applied particle swarm optimization on n-Queens problem.

Lack of information about details of the problem made genetic algorithm confused in searching state space of the problem. In this paper, we attempt to resolve this weakness with the help of local search methods. For this purpose, we use 'minimal conflicts algorithm' as a local search algorithm [9]. After next step has been chosen by genetic algorithm, minimal conflicts algorithm, as a secondary search, look at the adjacent states of the chosen step, to replace it with a better one.

The rest of the paper is organized as follows: section II reviews n-Queens problem. Section III describes minimal conflicts algorithm. Section IV briefly explains implementation of genetic algorithm. Section V describes modification of genetic algorithm. Section VI discusses the experimental results and eventually section VII concludes the paper.

II. N-QUEENS PROBLEM

In previous section, general definition of problem is stated. This problem contains three constraints: 1st, no two queens can share a same row. 2nd, no two queens can share a same column. 3rd, no two queens can share a same diameter.

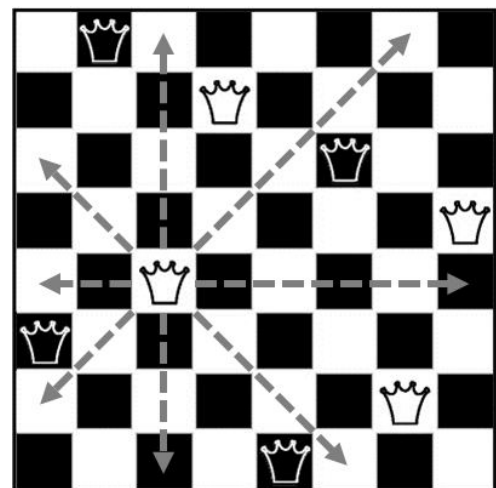


Figure 1: territory of each queen is row, column and diameters of its position on the chessboard.

Decision variable of this problem is a one-dimensional array of length n . Every permutation of possible values of the decision variables, presents a state of search-space of the problem. Definition of decision variable satisfies the second constraint:

$$A = \{(Q_1, Q_2, \dots, Q_n) | Q_i \in \{1, 2, \dots, n\}\} \quad (1)$$

Where A is decision variable and Q_i is the i th cell of decision variable, corresponding i th column of chessboard, containing the row of queen's location. First constraint is expressed as follows:

$$\forall i, j \in \{1, \dots, n\}, Q_i \neq Q_j \quad (2)$$

Third constraint is also expressed as follows:

$$\forall i, j \in \{1, \dots, n\}, |Q_i - Q_j| \neq |i - j| \quad (3)$$

III. MINIMAL CONFLICTS ALGORITHM

The role of 'minimal conflicts algorithm' in improving genetic algorithm was described in introduction section. According to Minton and his colleagues in [9] this algorithm has good performance in n -Queens problem.

Each state of search-space of the problem can be a candidate for solution. To remember, each cell of decision variable's array corresponds to a column of chessboard. This algorithm moves along candidate's array and by reaching to each column which its queen is in conflict with the other queens, tries to place it in a better row. If there is more than one location with least conflicts (= have more than one choices) one of them is selected, randomly. Eventually the result of this operation led to reducing conflicts on entire chessboard. Figure 2 shows the algorithm's pseudo code.

In Figure 3, an example of how algorithm works is illustrated. In this figure, the algorithm is looking at the third column of chessboard. In part 'a', the queen belongs to the seventh row is dealing with three conflicts. Then the algorithm assesses the

```

solution ← greedy-initializing(n)
for i = 1 : n
    if conflict-count (solution[i]) ≥ 1
        for walk = 1 : n
            solution[i] ← walk
            col-c-count[walk] ← conflict-count (solution[i])
        end of inner for
        best-place ← min(col-c-count[walk])
        solution[i] ← best-place
    end of if
end of outter for

```

col-c-count[]: An array that contains number of conflicts for each row in a considered column i .
conflict-count(): A function which count number of conflicts that a queen has with the others.

Figure 2: pseudo code for Minimal conflicts algorithm

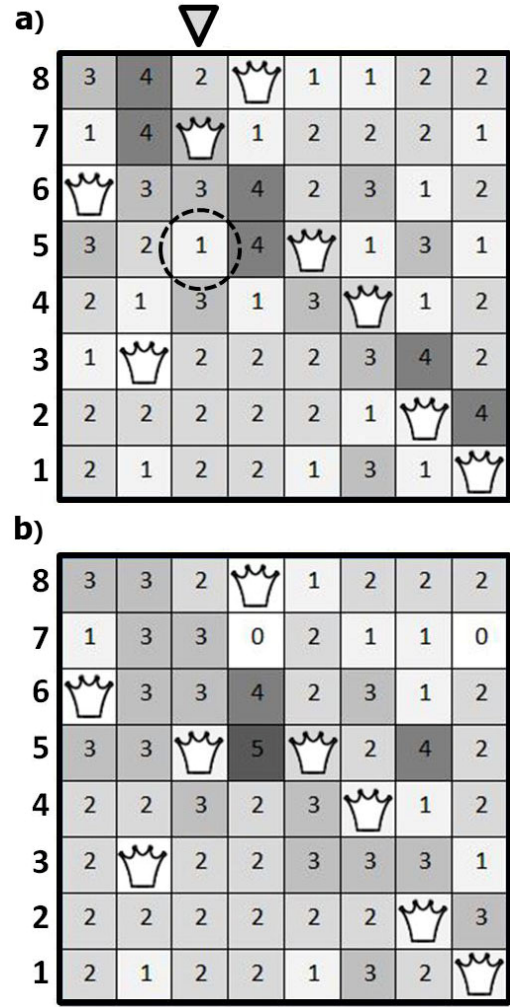


Figure 3: Part 'a' shows the situation of chessboard before applying the algorithm and part 'b' shows the situation after it. Values located in each cells present the number of conflicts, if queen was placed in that cell. Cells with a fewer conflicts, have been painted with a lighter color and cells with a more conflicts, have been painted with a darker color.

situation of other cells in the third column, in order to find a better place for the queen. Finally it concludes that fifth row with, just one conflict is a better place for the queen. This change has been applied in part 'b'.

IV. GENETIC ALGORITHM

In different references, genetic algorithm has the same concept but various implementations. In this paper, we implement this algorithm based on description that Whitley presents in [1].

As it is mentioned before, each permutation of possible values of the decision variable can be a candidate to problem's solution. These candidates are also called 'chromosomes'. A collection of candidates are called 'population'. Genetic algorithm is consisted of several operators. Applying these operators cause population modification and during these modifications new generations are created.

Figure 4 shows that in this implementation of genetic algorithm, each iteration is divided into 'selection phase' and 'recombination phase'.

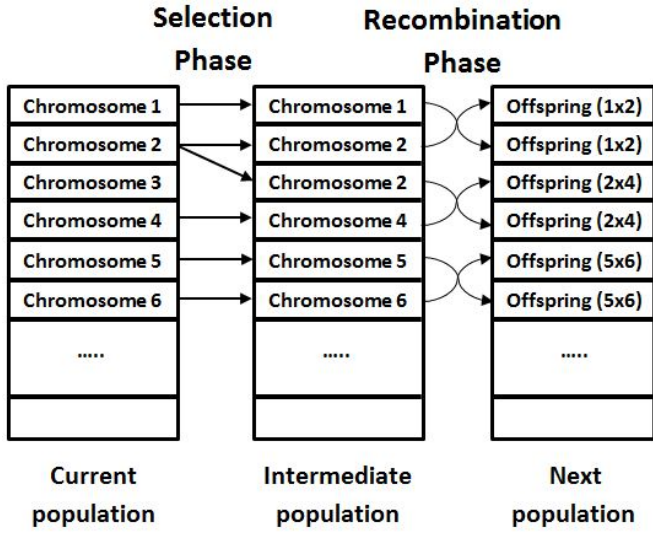


Figure 4: each generation is modified during selection and recombination phase and creates new generation of candidates [1].

Fitness function is measure of distance or proximity of a candidate for problem's solution which is shown by $f(x)$. At the beginning of selection phase, current-population is initialized by initial-population, at the first iteration. In other iterations it is initialized by next-population belongs to previous iteration. Then, fitness function assesses candidates and each candidate will be labeled by fitness-value which shows how candidate is close to the answer. If the answer is found then the algorithm stops but if the answer is not found, intermediate-population would be composed based on current-population, during selection operation. Among the algorithms that are used for selection operation, 'roulette wheel' and 'remainder stochastic sampling' are more significant [1]. In this paper we used 'roulette wheel' technique for selection operation. The probability of being selected through roulette wheel is calculated from the following formula:

$$P_i = f_i / \sum_{i=1}^k f_i \quad (4)$$

f_i is the fitness function and k is the size of population. Before continuing, intermediate-population must be shuffled, sufficiently.

During recombination phase, next-population is created by applying 'crossover' and 'mutation' on candidates from intermediate-population. Crossover operator chooses a pair of candidates. Then it recombines them with the probability P_c to form two new candidates. Crossover operator has various types like: 1-point crossover, 2-point crossover, etc [1].

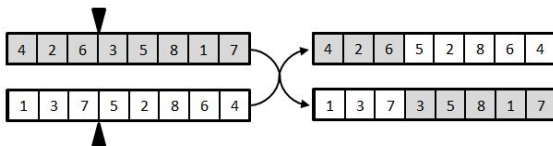


Figure 5: 1-point crossover cuts candidate's arrays from 'break point' then it replaces primary or secondary pieces.

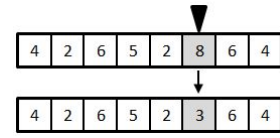


Figure 6: Mutation operation.

After crossover operation has been carried out, we can apply mutation operation. Each value from candidates in the population, can be changed under the probability P_m .

V. MODIFIED GENETIC ALGORITHM

States which have better fitness-value are more likely adjacent to one of the answers of problem. As we mentioned before, at the end of iteration, genetic algorithm presents a population of candidates which might have consist the answers of problem. The role of minimal conflicts algorithm is to replace each of these candidates with a better one by searching adjacent states. This algorithm manages a sub-search under iteration of genetic algorithm.

Figure 7 represents the process of iteration of modified genetic algorithm. Minimal conflicts algorithm is looking at adjacent space of each candidate and trying to replace current candidate by one of its neighbors which has a better fitness-value.

In previous section, we mentioned that genetic algorithm consists of several operators which are applied in iterative order. In Modified genetic algorithm, minimal conflicts algorithm is applied to candidates beside crossover and mutation, as an additional operator.

After hybridizing genetic algorithm and minimal conflicts algorithm as it described above, we are trying to increase algorithm's speed of reaching to answer with the help of two following techniques:

A. Greedy Initialization instead of Random initialization

To remember, initializing population is especially important in genetic algorithm and has a significant impact on its efficiency.

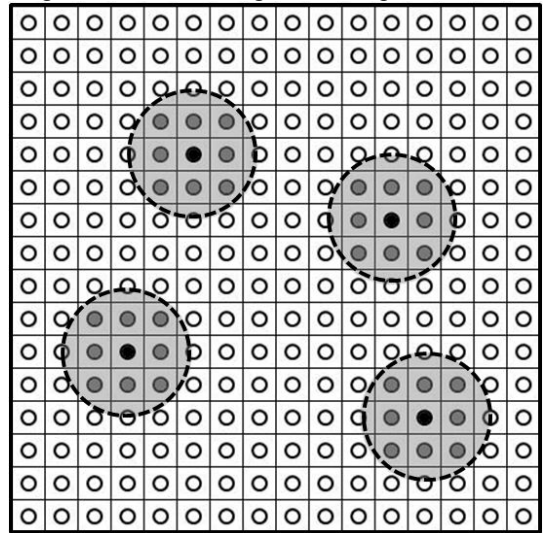


Figure 7: Above board, is a symbolic representation of state space of 4-Queens problem. Black dots representation candidates and gray circles highlights adjacent states of each candidate.

Before the first iteration begins, initial-population is assigned using greedy algorithm which iterates through columns and locates each queen on the row that has the least conflicts with other queens which previously placed. If there is more than one location with least conflicts (= have more than one choices) one of them is selected, randomly.

B. Crossover with best break-point

In situation where break-point is selected randomly, candidates resulting from crossover operation (= offspring candidates) might be better or worse than their parents. But if we look at the results of all possible break-points and choose the best one, each generation will always equal or better than previous generation.

In Figure 8, gray areas represent modified sections of standard genetic algorithm. Minimal conflicts operator applied to each candidates after crossover and mutation as a third operator of 'recombination phase'.

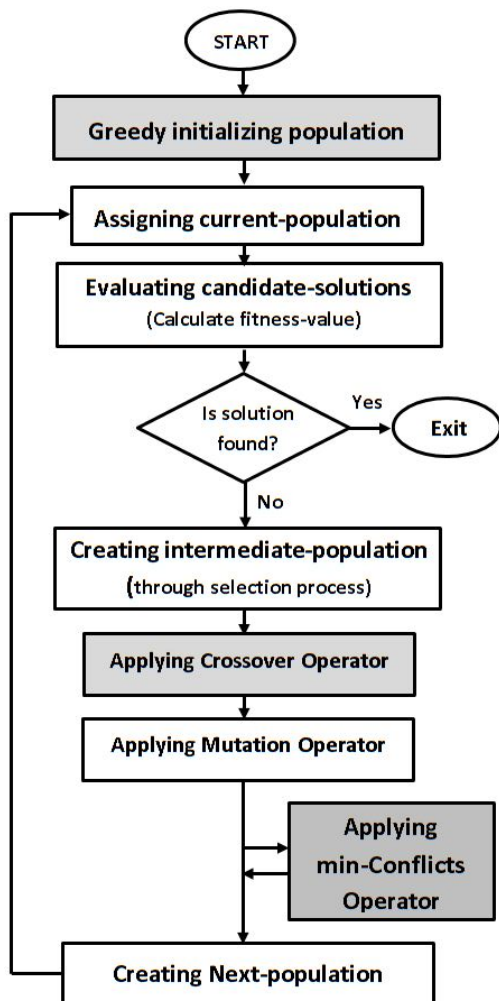


Figure 8: flowchart of Modified genetic algorithm

The algorithm's pseudo code will be as follows:

```

Initial-population ← greedy-initializing(n)
current-population ← Initial-population
for iter = 1 : maximum-iteration
    for i=1: k
        fitness-value[i]←fitness-function(current-population[i])
    end of inner for
    if solution is found
        break;
    end of if
    intermediate-population←selection(current-population)
    temp ← crossover(intermediate-population)
    temp ← mutation(temp)
    temp ← minimal-conflicts-algorithm(temp)
    next-population ← temp
    current-population ← n_population
end of for
  
```

k : population size

Figure 9: pseudo code for Modified genetic algorithm

VI. EXPERIMENTAL RESULTS

To ensure that performance of 'modified genetic algorithm' is as efficient as we expected, we need to test it. We can assess the amount of improved efficiency by comparing the results of 'modified genetic algorithm' with the results of 'standard genetic algorithm'.

To prevent accidental results, algorithms run a hundred times, for each n . The upper-bound for iterations is considered equal to $50 \times n$. If the number of iterations in a run of algorithm is exceeded the limit, then the result is a failure and its number of iteration is considered equal to the upper-bound. Population size for 'standard genetic algorithm' is equal to $25 * n$ and for 'modified genetic algorithm' is equal to $n/2$. According to [10], probability of crossover is equal to 0.7 ($P_c = 0.7$) and according to [1], 0.01 is suitable value for probability of mutation ($P_m = 0.01$).

Table 1: Comparing the results based on the average number of iterations

n	Hybrid Genetic Algorithm	Standard Genetic Algorithm
n = 8	4.78 {1-89}	242.61 (47%) {2-400}
n = 16	5.41 {1-34}	534.84 (42%) {8-800}
n = 32	3.81 {1-13}	863.83 (65%) {24-1600}
n = 64	3.05 {1-8}	964.86 (92%) {81-3200}
n = 128	2.74 {1-5}	1192 (98%) {212-6400}
n = 256	2.43 {1-4}	*
n = 512	2.51 {1-3}	*

Table 1 represents the comparison between ‘modified genetic algorithm’ and ‘standard genetic algorithm’ based on their average number of iterations.

When some runs of algorithm associated with failure, value in parentheses besides corresponding table entry, indicates the percentage of time which algorithm successfully completed. Minimal and maximal numbers of iterations are also placed in brackets beside each table entries.

Comparing the results of second and third columns of Table 1 shows that ‘modified genetic algorithm’ successfully completed in all runs but standard genetic algorithm contains failure. The result in Table 1 shows that ‘modified genetic algorithm’, regardless of size of problem, reaches to answer in approximately constant average number of iterations (approximately 3 iterations). But average number of iterations for standard genetic algorithm increases non-linearly according to size of the problem.

Beside ‘average number of iterations’ we need another criterion which can assess computational complexity of algorithms. As a computational complexity criteria, ‘Average times of evaluating fitness function’ is a suitable because it is independent from hardware and implementation quality. The results of comparison based on this criterion is presented in Table 2.

Comparing the results of second and third columns of Table 2 shows that in ‘modified genetic algorithm’ the amount of computation is decreased in compared to ‘standard genetic algorithm’. Also modified genetic algorithm has additional computational complexity due to minimal conflicts operator but ‘large number of iterations’ and ‘large population size’, extremely increases the ‘average times of evaluating fitness function’.

VII. CONCLUSION

Considering that standard genetic algorithm is not efficient enough in solving large scales of n-Queens problem, in this paper we attempt to resolve weakness of genetic algorithm by using minimal conflicts algorithm. At each iteration of genetic algorithm’s process, minimal conflicts algorithm try to replace candidate-solutions by a better one. This can accelerate genetic algorithm in order to find the problem’s answer, quicker. Also we can help genetic algorithm to avoid wrong steps by using

‘greedy initializing’ and ‘best break-point’ techniques. According to the results which are declared in section 6, the modifications applied to genetic algorithm improved its efficiency in solving n-Queens problem.

REFERENCES

- [1] D. Whitley, "a genetic algorithm tutorial," *statistics and computing*, pp. 65-85, 1995
- [2] J. Bell, B. Stevens, "a survey of known results and research areas for n-queens," *Discrete Mathematics* 309, pp. 1-31, 2009.
- [3] Ivica Martinjak and M. Golub, "comparison of heuristic algorithms for the n-Queen Problem," in *international conference on information technology Interfaces*, 2007.
- [4] R. G. Sharma, B. Keswani, "implementation of n-queens puzzle using meta-heuristic algorithm (cuckoo search)," in *International Journal of Latest Trends in Engineering and Technology*, 2013.
- [5] J. Turner, A. Homaifar, S. Ali, "the n-queens problem and genetic algorithms," in *IEEE*, 1992.
- [6] F. Meng, Sh. Wu., "research of hybrid genetic algorithm in n-queen problem based on HCI," in *IEEE*, 2008.
- [7] M. Boikovic, M. Golub, L. Budin, "solving n-Queen problem using global parallel genetic algorithm," in *IEEE*, 2003.
- [8] A. Amooshahi, M. Joudaki, M. Imani, N. Mazhari, "presenting a new method based on cooperative PSO to solve permutation problems: A case study of n-queen problem," in *IEEE*, 2011.
- [9] S. Minton, M. D. Johnston, A. B. Philips and P. Laird, "Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems," *Artificial Intelligence* 58, pp. 161-205, 1992.
- [10] M. Negnevitsky, *artificial intelligence: a guide to intelligent systems* (2nd edition), Addison Wesley press, 2005.

Table 2: Comparing the results based on the average times of evaluating fitness function

n	Hybrid Genetic Algorithm	Standard Genetic Algorithm
n = 8	554.48	48522.00
n = 16	2466.96	213936.00
n = 32	6888.48	691064.00
n = 64	21960.00	1543776.00
n = 128	78736.64	3814400.00
n = 256	279002.90	*
n = 512	1152110.00	*